ONLINE EXAMINATION PORTAL

BCA Final-Year Project Report

Technology: Python Flask, SQLite, HTML, CSS, Bootstrap, JavaScript

Prepared as an academic software project

Student Name: _____

College / University: _____

Academic Session: _____

1. Abstract

The Online Examination Portal is a web-based system designed to support the creation, delivery, and evaluation of objective examinations. Administrators and teachers can create exams, add multiple-choice questions, set a duration, and publish exams. Students can register, log in, attempt published exams within a countdown timer, and review results after submission. The backend uses Python Flask and SQLite. Passwords are stored as secure hashes, and MCQ scoring is performed on the server.

2. Problem Statement

Paper-based examination workflows can require manual question distribution, checking, score calculation, and result preparation. This project demonstrates a centralized portal that reduces repetitive work for objective tests and gives students access to their results through a browser.

3. Objectives

• Provide role-based access for administrator, teacher, and student users.

• Allow staff to create exams and manage multiple-choice questions.

• Provide timed attempts and automatic submission when the deadline is reached.

• Evaluate objective answers on the server.

• Display result history and export staff results as CSV.

• Store users, exams, questions, attempts, and answers in SQLite.

4. Scope

The project is a local academic prototype suitable for demonstration and further development. It supports objective MCQs and a single attempt per student per exam. It does not currently include essay marking, online proctoring, email verification, password recovery, or production-grade deployment controls.

5. Technology Used

Frontend: HTML, CSS, Bootstrap 5, JavaScript. Backend: Python with Flask. Database: SQLite. Authentication: Flask sessions and Werkzeug password hashing. Reporting: CSV export. Tests: Python unittest and Flask test client.

6. System Roles

Administrator: views dashboards, creates teacher/student accounts, and manages exams. Teacher: creates exams, adds questions, publishes/unpublishes exams, reviews results, and exports a CSV. Student: registers, views published exams, starts or resumes an unfinished attempt, submits answers, and reviews submitted results.

7. Functional Requirements

• Registration and login with role checks.

• Exam creation with title, description, and duration.

• Question creation with four options, a correct option, and marks.

• Publish/unpublish exam workflow.

• Timed student attempt and server-side answer saving.

• Automatic MCQ score calculation.

• Result review and CSV export.

8. Non-Functional Requirements

The interface should be understandable and responsive. Passwords should not be stored in plaintext. Role restrictions should prevent students from accessing staff functions. Input should be validated on the server. A production deployment should add CSRF protection, rate limiting, stronger secret management, monitoring, and server-side expiry handling.

9. Database Design

users(id, name, email, password_hash, role, created_at); exams(id, title, description, duration_minutes, is_published, created_by, created_at); questions(id, exam_id, question_text, option_a, option_b, option_c, option_d, correct_option, marks); attempts(id, exam_id, student_id, started_at, deadline_at, submitted_at, score, total_marks); answers(id, attempt_id, question_id, selected_option). Foreign keys connect exams to their creator, questions to exams, attempts to students/exams, and answers to attempts/questions.

10. System Workflow

Staff logs in and creates a draft exam. Staff adds questions and publishes it. A student logs in and starts an available exam. The server creates an attempt with a deadline and initializes answer rows. Selected options are sent to the backend. On manual submission or timer expiry, the server compares saved selections with the answer key, stores the score and submission time, and displays the result.

11. Testing Plan

1. Open the home page and confirm HTTP 200.

2. Log in with each demo role.

3. Confirm students receive HTTP 403 for admin-only user management.

4. Create an exam, add questions, and publish it.

5. Log in as a student and take the exam.

6. Verify selected answers save and submitted score is correct.

7. Confirm completed exams cannot be retaken under the one-attempt rule.

8. Export results CSV as staff.

9. Run `python -m unittest discover -s tests -v`.

12. Setup Instructions

Install Python 3.10 or newer. In the project directory, create and activate a virtual environment, run `python -m pip install -r requirements.txt`, and then run `python app.py`. Open http://127.0.0.1:5000. The SQLite database and sample exam are initialized automatically. Demo credentials are listed in README.md and must be changed before deployment.

13. Limitations and Future Enhancements

Possible improvements include configurable pass marks, password reset, email notifications, question randomization, negative marking, PDF result reports, multiple attempts, import/export of questions, accessibility testing, CSRF protection, a server-side expiry worker, and deployment with a production WSGI server.

14. Conclusion

The Online Examination Portal demonstrates a complete web application workflow: authentication, role-based access, CRUD operations, database persistence, timed interaction, server-side scoring, and result reporting. It is a suitable academic foundation that can be extended into a more robust examination system.

Appendix: Demo accounts (local development only)

Role	Email	Password
Admin	admin@example.com	Admin@123
Teacher	teacher@example.com	Teacher@123
Student	student@example.com	Student@123

Security note: demo credentials are for local testing only. Set a strong SECRET_KEY, change demo passwords, and add production security controls before any public deployment.